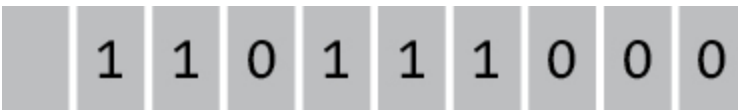


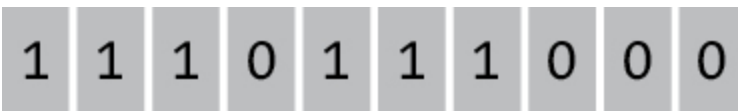
[Figure 5-4](#): A string of 10 bits

After starting with these bits, we can proceed through a simple LFSR algorithm. We start by calculating a simple sum of a subset of the bits—for example, the sum of the 4th bit, 6th bit, 8th bit, and 10th bit (we could also choose other subsets). In this case, that sum is 3. Our computer architecture can only store 0s and 1s, so we take our sum mod 2, and end up with 1 as our final sum. Then we remove our rightmost bit and shift every remaining bit one position to the right ([Figure 5-5](#)).



[Figure 5-5](#): Bits after removal and shifting

Since we removed a bit and shifted everything, we have an empty space where we should insert a new bit. The bit we insert here is the sum we calculated before. After that insertion, we have the new state of our bits ([Figure 5-6](#)).



[Figure 5-6](#): Bits after replacement with a sum of selected bits

We take the bit we removed from the right side as the output of the algorithm, the pseudorandom number that this algorithm is supposed to generate. And now that we have a new set of 10 ordered bits, we can run a new round of the algorithm and get a new pseudorandom bit just as before. We can repeat this process as long as we'd like.

In Python, we can implement a feedback shift register relatively simply. Instead of directly overwriting individual bits on the hard drive, we will just create a list of bits like the following:

---

```
bits = [1,1,1]
```

---